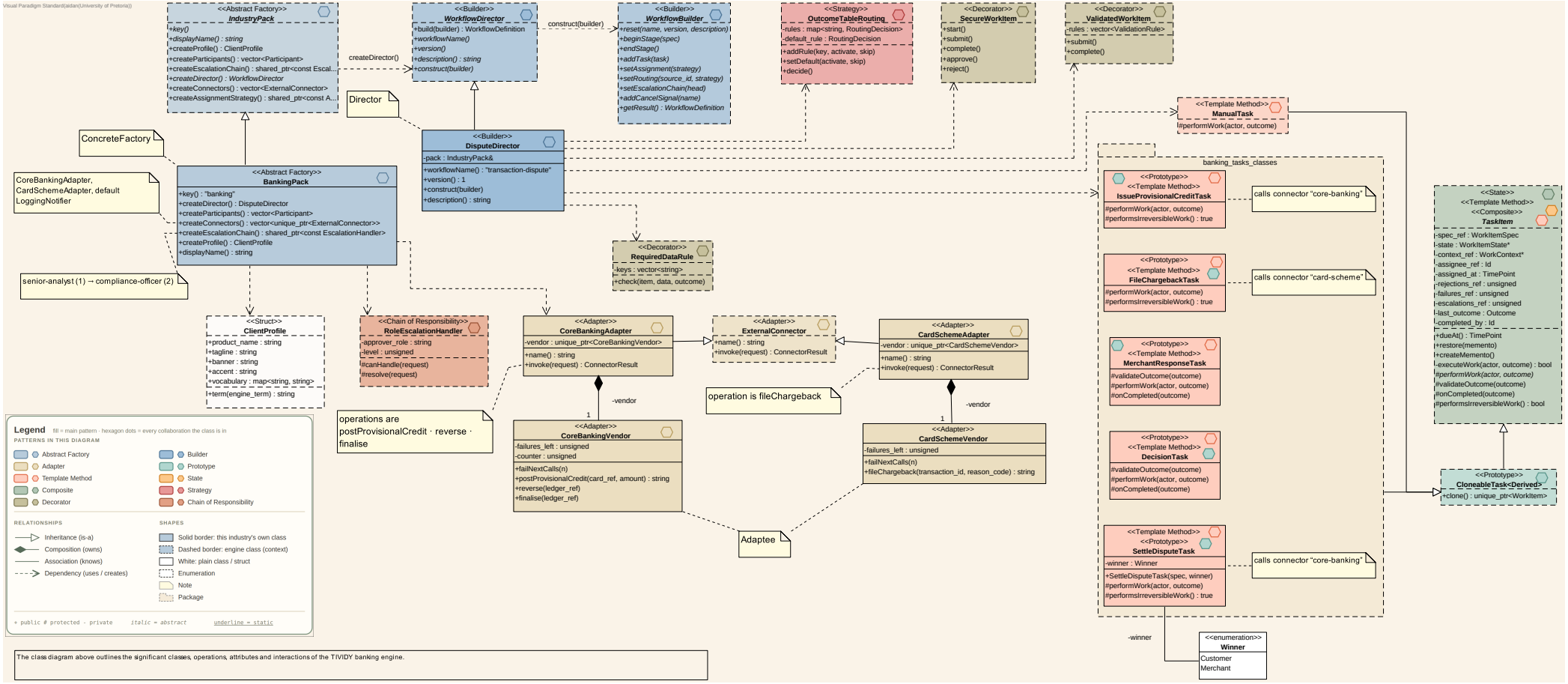
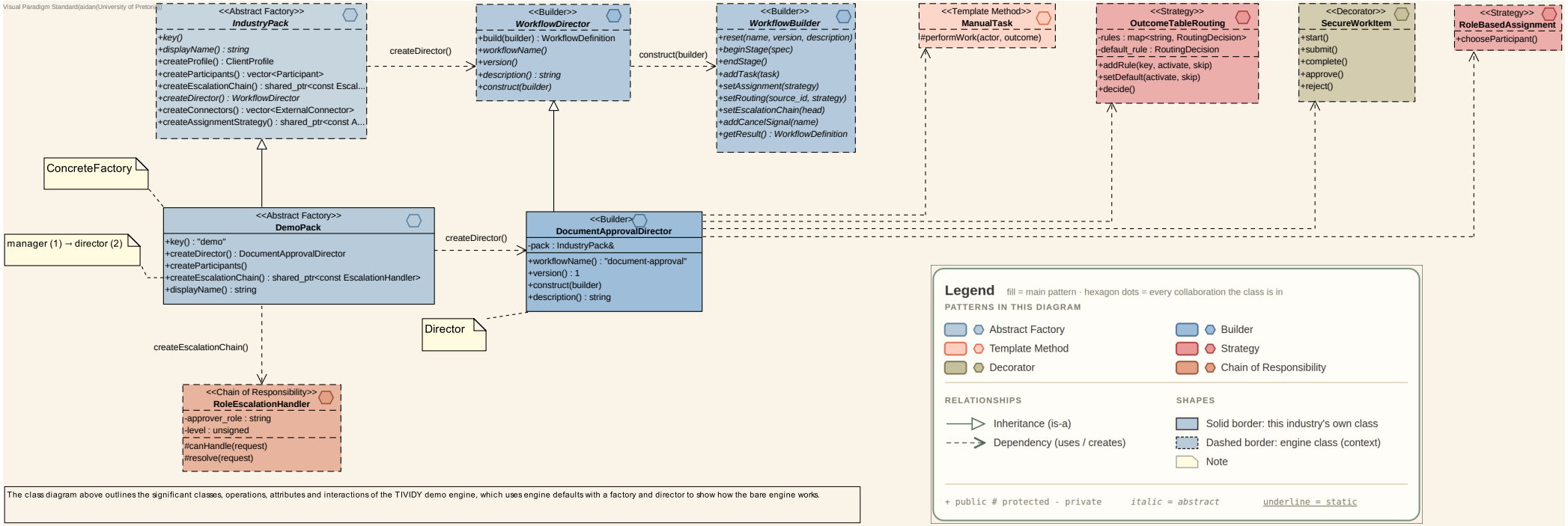
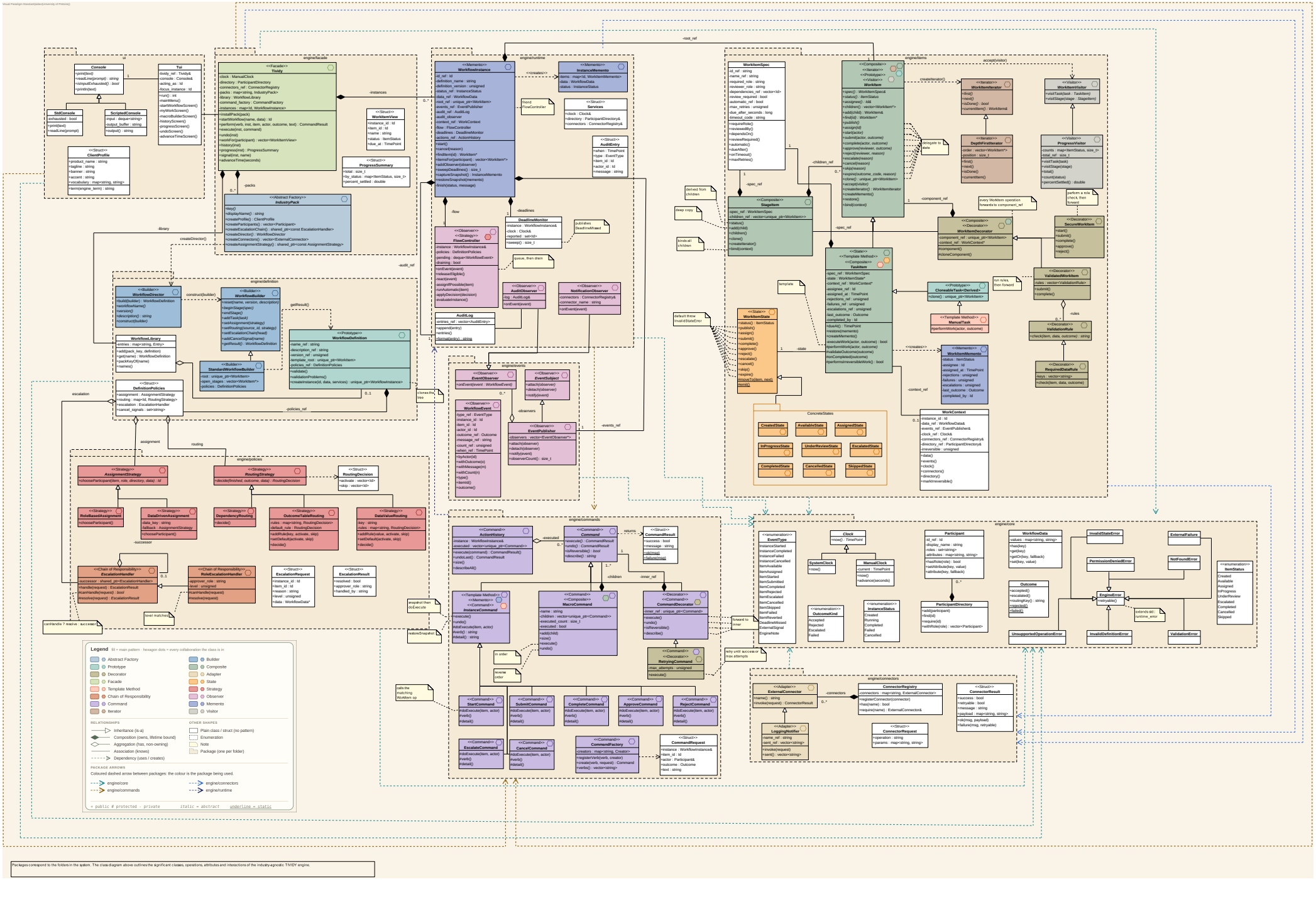
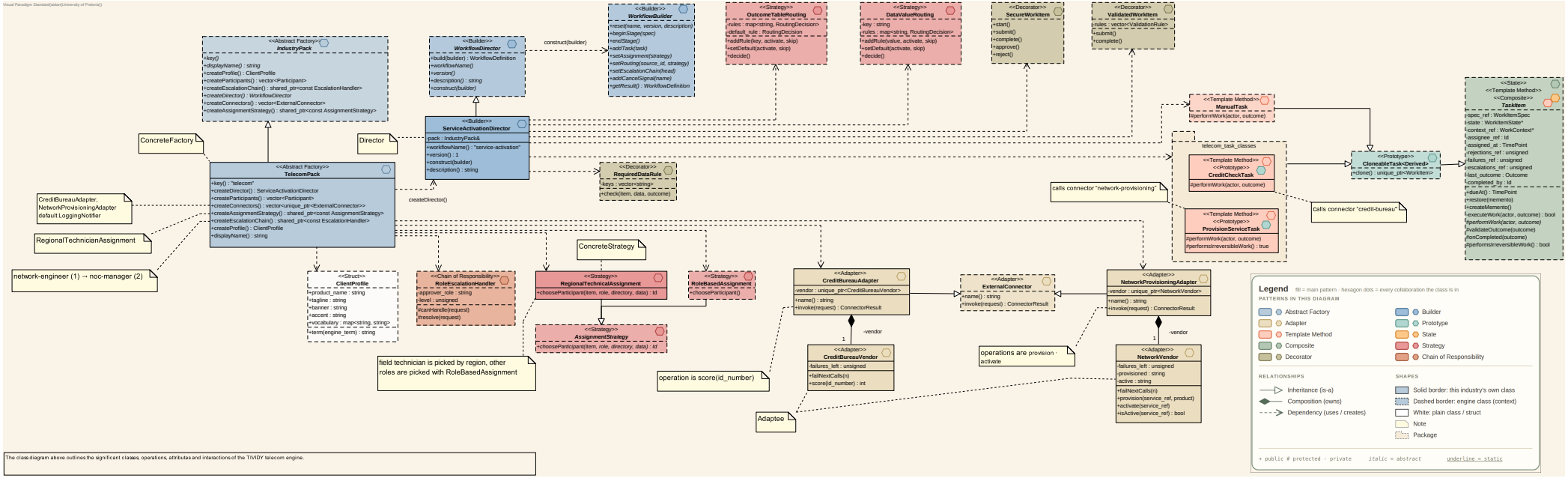
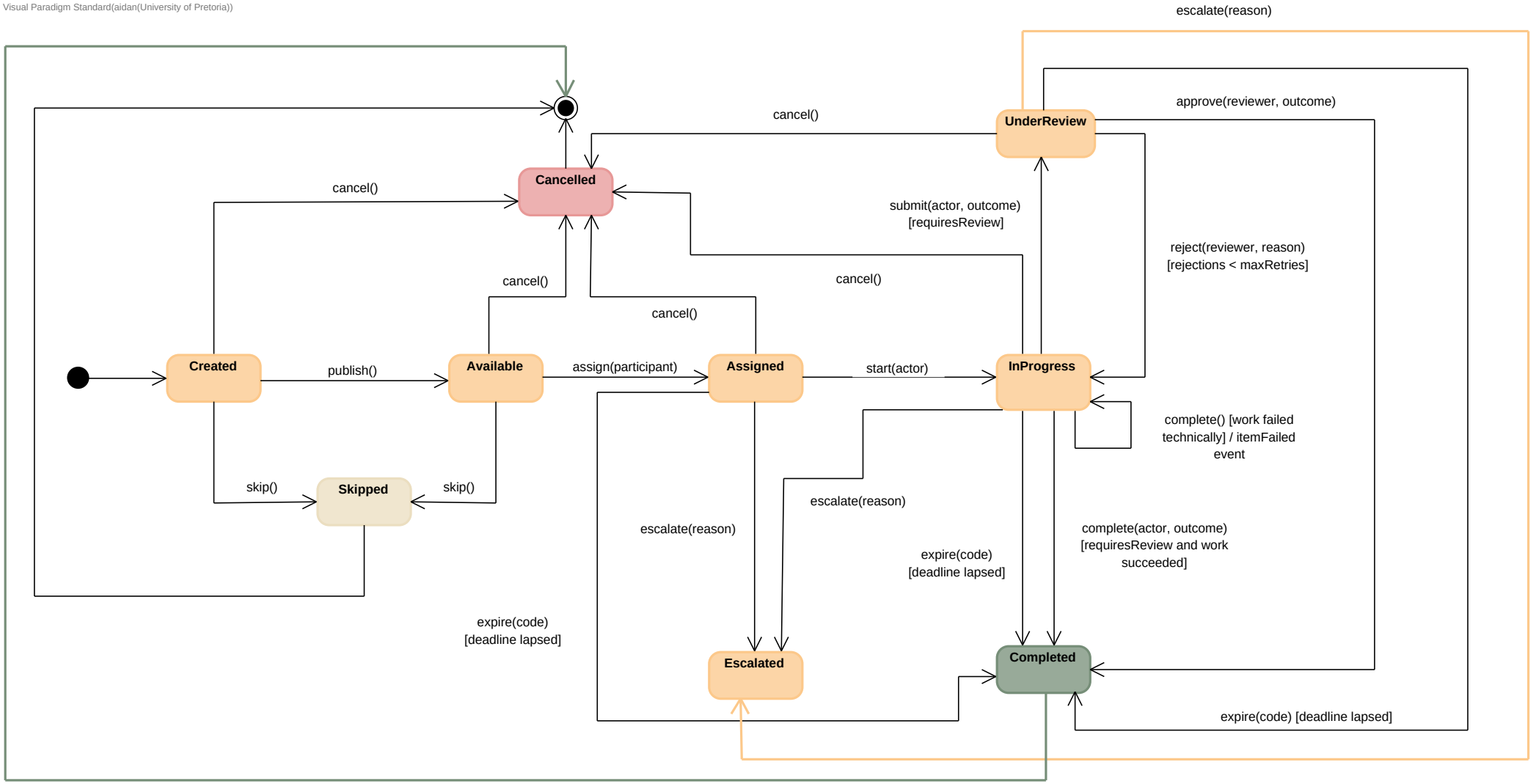

Visual Paradigm Standard(aidan(University of Pretoria))
<<Abstract Factory>>
IndustryPack
+key()
+displayName() : string
+createProfile() : ClientProfile
+createParticipants() : vector<Participant>
+createEscalationChain() : shared_ptr<const Escal...
+createDirector() : WorkflowDirector
+createConnectors() : vector<ExternalConnector>
+createAssignmentStrategy() : shared_ptr<const A...
createDirector()
<<Builder>>
WorkflowDirector
+build(builder) : WorkflowDefinition
+workflowName()
+version()
+description() : string
+construct(builder)
construct(builder)
<<Builder>>
WorkflowBuilder
+reset(name, version, description)
+beginStage(spec)
+endStage()
+addTask(task)
+setAssignment(strategy)
+setRouting(source_id, strategy)
+setEscalationChain(head)
+addCancelSignal(name)
+getResult() : WorkflowDefinition
<<Strategy>>
OutcomeTableRouting
-rules : map<string, RoutingDecision>
-default_rule : RoutingDecision
+addRule(key, activate, skip)
+setDefault(activate, skip)
+decide()
<<Decorator>>
SecureWorkItem
+start()
+submit()
+complete()
+approve()
+reject()
<<Decorator>>
ValidatedWorkItem
-rules : vector<ValidationRule>
+submit()
+complete()
Director
ConcreteFactory
<<Builder>>
DisputeDirector
-pack : IndustryPack&
+workflowName() : "transaction-dispute"
+version() : 1
+construct(builder)
+description() : string
<<Template Method>>
ManualTask
#performWork(actor, outcome)
CoreBankingAdapter, CardSchemeAdapter, default LoggingNotifier
<<Abstract Factory>>
BankingPack
+key() : "banking"
+createDirector() : DisputeDirector
+createParticipants() : vector<Participant>
+createConnectors() : vector<unique_ptr<ExternalConnector>>
+createEscalationChain() : shared_ptr<const EscalationHandler>
+createProfile() : ClientProfile
+displayName() : string
<<Decorator>>
RequiredDataRule
-keys : vector<string>
+check(item, data, outcome)
banking_tasks_classes
<<Prototype>>
<<Template Method>>
IssueProvisionalCreditTask
#performWork(actor, outcome)
#performsIrreversibleWork() : true
calls connector "core-banking"
<<Prototype>>
<<Template Method>>
FileChargebackTask
#performWork(actor, outcome)
#performsIrreversibleWork() : true
calls connector "card-scheme"
<<Prototype>>
<<Template Method>>
MerchantResponseTask
#validateOutcome(outcome)
#performWork(actor, outcome)
#onCompleted(outcome)
<<Prototype>>
<<Template Method>>
DecisionTask
#validateOutcome(outcome)
#performWork(actor, outcome)
#onCompleted(outcome)
<<Template Method>>
<<Prototype>>
SettleDisputeTask
-winner : Winner
+SettleDisputeTask(spec, winner)
#performWork(actor, outcome)
#performsIrreversibleWork() : true
calls connector "core-banking"
<<State>>
<<Template Method>>
<<Composite>>
TaskItem
-spec_ref : WorkItemSpec
-state : WorkItemState*
-context_ref : WorkContext*
-assignee_ref : Id
-assigned_at : TimePoint
-rejections_ref : unsigned
-failures_ref : unsigned
-escalations_ref : unsigned
-last_outcome : Outcome
-completed_by : Id
+dueAt() : TimePoint
+restore(memento)
+createMemento()
+executeWork(actor, outcome) : bool
#performWork(actor, outcome)
#validateOutcome(outcome)
#onCompleted(outcome)
#performsIrreversibleWork() : bool
<<Prototype>>
CloneableTask<Derived>
+clone() : unique_ptr<WorkItem>
senior-analyst (1) → compliance-officer (2)
<<Struct>>
ClientProfile
+product_name : string
+tagline : string
+banner : string
+accent : string
+vocabulary : map<string, string>
+term(engine_term) : string
<<Chain of Responsibility>>
RoleEscalationHandler
-approver_role : string
-level : unsigned
#canHandle(request)
#resolve(request)
<<Adapter>>
CoreBankingAdapter
-vendor : unique_ptr<CoreBankingVendor>
+name() : string
+invoke(request) : ConnectorResult
<<Adapter>>
ExternalConnector
+name() : string
+invoke(request) : ConnectorResult
<<Adapter>>
CardSchemeAdapter
-vendor : unique_ptr<CardSchemeVendor>
+name() : string
+invoke(request) : ConnectorResult
-vendor
1
<<Adapter>>
CoreBankingVendor
-failures_left : unsigned
-counter : unsigned
+failNextCalls(n)
+postProvisionalCredit(card_ref, amount) : string
+reverse(ledger_ref)
+finalise(ledger_ref)
operations are postProvisionalCredit · reverse · finalise
operation is fileChargeback
-vendor
1
<<Adapter>>
CardSchemeVendor
-failures_left : unsigned
+failNextCalls(n)
+fileChargeback(transaction_id, reason_code) : string
Adaptee
-winner
<<enumeration>>
Winner
Customer
Merchant
Legend
fill = main pattern · hexagon dots = every collaboration the class is in
PATTERNS IN THIS DIAGRAM
Abstract Factory
Adapter
Template Method
Composite
Decorator
Builder
Prototype
State
Strategy
Chain of Responsibility
RELATIONSHIPS
Inheritance (is-a)
Composition (owns)
Association (knows)
Dependency (uses / creates)
SHAPES
Solid border: this industry's own class
Dashed border: engine class (context)
White: plain class / struct
Enumeration
Note
Package
+ public # protected - private
italic = abstract
underline = static
The class diagram above outlines the significant classes, operations, attributes and interactions of the TIVIDY banking engine.

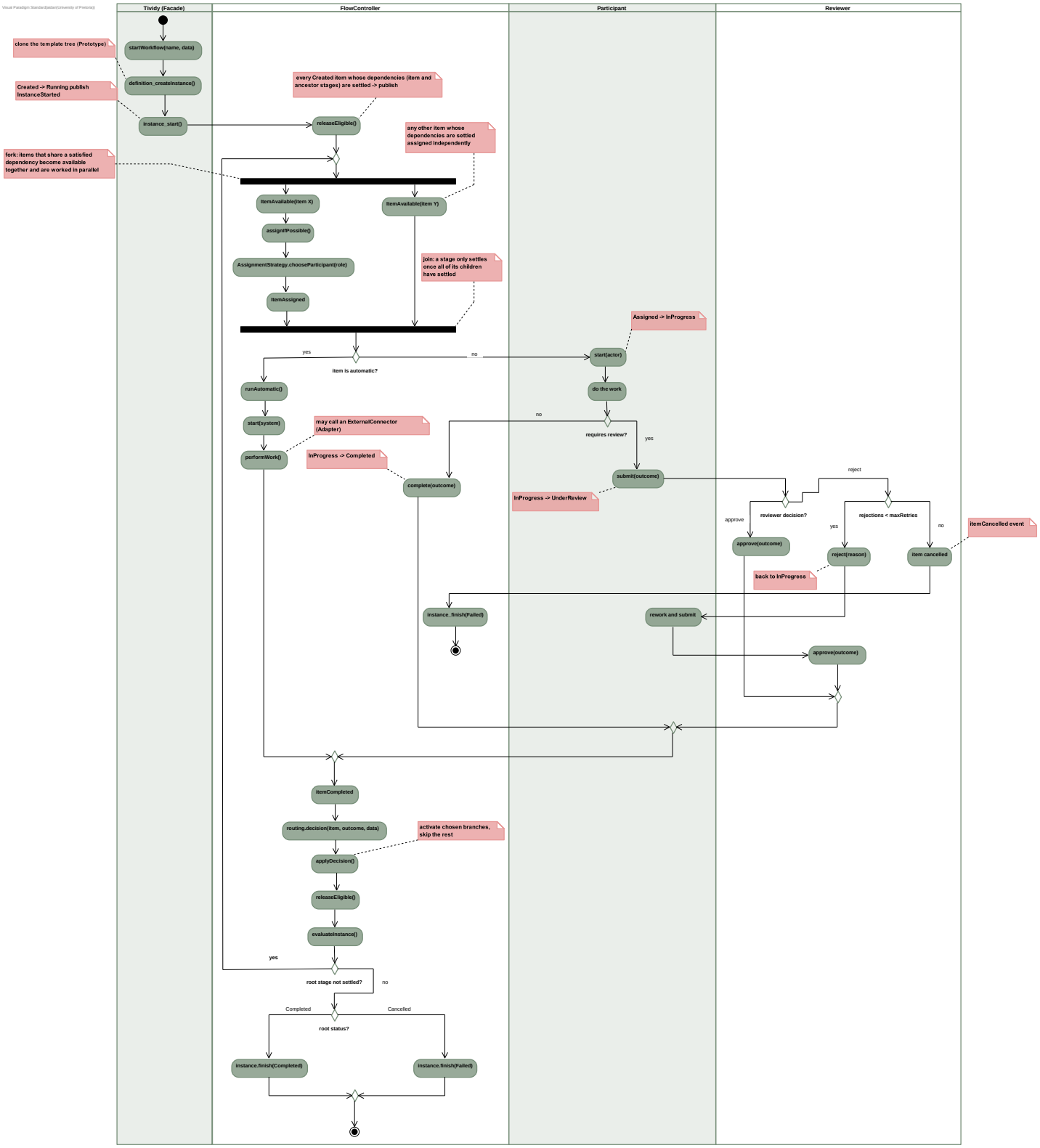

Visual Paradigm Standard(aidan(University of Pretoria))
<<Abstract Factory>>
IndustryPack
+key()
+displayName() : string
+createProfile() : ClientProfile
+createParticipants() : vector<Participant>
+createEscalationChain() : shared_ptr<const Escal...
+createDirector() : WorkflowDirector
+createConnectors() : vector<ExternalConnector>
+createAssignmentStrategy() : shared_ptr<const A...
createDirector()
<<Builder>>
WorkflowDirector
+build(builder) : WorkflowDefinition
+workflowName()
+version()
+description() : string
+construct(builder)
construct(builder)
<<Builder>>
WorkflowBuilder
+reset(name, version, description)
+beginStage(spec)
+endStage()
+addTask(task)
+setAssignment(strategy)
+setRouting(source_id, strategy)
+setEscalationChain(head)
+addCancelSignal(name)
+getResult() : WorkflowDefinition
<<Template Method>>
ManualTask
#performWork(actor, outcome)
<<Strategy>>
OutcomeTableRouting
-rules : map<string, RoutingDecision>
-default_rule : RoutingDecision
+addRule(key, activate, skip)
+setDefault(activate, skip)
+decide()
<<Decorator>>
SecureWorkItem
+start()
+submit()
+complete()
+approve()
+reject()
<<Strategy>>
RoleBasedAssignment
+chooseParticipant()
ConcreteFactory
<<Abstract Factory>>
DemoPack
+key() : "demo"
+createDirector() : DocumentApprovalDirector
+createParticipants()
+createEscalationChain() : shared_ptr<const EscalationHandler>
+displayName() : string
createDirector()
<<Builder>>
DocumentApprovalDirector
-pack : IndustryPack&
+workflowName() : "document-approval"
+version() : 1
+construct(builder)
+description() : string
manager (1) → director (2)
Director
createEscalationChain()
<<Chain of Responsibility>>
RoleEscalationHandler
-approver_role : string
-level : unsigned
#canHandle(request)
#resolve(request)
The class diagram above outlines the significant classes, operations, attributes and interactions of the TIVIDY demo engine, which uses engine defaults with a factory and director to show how the bare engine works.
Legend
fill = main pattern · hexagon dots = every collaboration the class is in
PATTERNS IN THIS DIAGRAM
Abstract Factory
Builder
Template Method
Strategy
Decorator
Chain of Responsibility
RELATIONSHIPS
Inheritance (is-a)
Dependency (uses / creates)
SHAPES
Solid border: this industry's own class
Dashed border: engine class (context)
Note
+ public # protected - private
italic = abstract
underline = static


Console
+print(text)
+readLine(prompt) : string
+isInputExhausted() : bool
+printIn(text)
StdConsole
-exhausted : bool
+print(text)
+readLine(prompt)
ScriptedConsole
-input : deque<string>
-output_buffer : string
+output() : string
Tui
-tividy_ref : Tividy&
-console : Console&
-acting_as : Id
-focus_instance : Id
+run() : int
-mainMenu()
-startWorkflowScreen()
-myWorkScreen()
-macroBuilderScreen()
-historyScreen()
-progressScreen()
-undoScreen()
-advanceTimeScreen()
<<Struct>>
ClientProfile
+product_name : string
+tagline : string
+banner : string
+accent : string
+vocabulary : map<string, string>
+term(engine_term) : string
engine/facade
<<Facade>>
Tividy
-clock : ManualClock
-directory : ParticipantDirectory
-connectors_ref : ConnectorRegistry
-packs : map<string, IndustryPack>
-library : WorkflowLibrary
-command_factory : CommandFactory
-instances : map<Id, WorkflowInstance>
+installPack(pack)
+startWorkflow(name, data) : Id
+perform(verb, inst, item, actor, outcome, text) : CommandResult
+execute(inst, command)
+undo(inst)
+workFor(participant) : vector<WorkItemView>
+history(inst)
+progress(inst) : ProgressSummary
+signal(inst, name)
+advanceTime(seconds)
-instances
-packs
<<Struct>>
WorkItemView
+instance_id : Id
+item_id : Id
+name : string
+status : ItemStatus
+due_at : TimePoint
<<Struct>>
ProgressSummary
+total : size_t
+by_status : map<ItemStatus, size_t>
+percent_settled : double
<<Abstract Factory>>
IndustryPack
+key()
+displayName() : string
+createProfile() : ClientProfile
+createParticipants() : vector<Participant>
+createEscalationChain() : shared_ptr<const EscalationHandler>
+createDirector() : WorkflowDirector
+createConnectors() : vector<ExternalConnector>
+createAssignmentStrategy() : shared_ptr<const AssignmentStrategy>
-packs
-library
createDirector()
-audit_ref
engine/definition
<<Builder>>
WorkflowDirector
-build(builder) : WorkflowDefinition
+workflowName()
+version()
+description() : string
+construct(builder)
construct(builder)
<<Builder>>
WorkflowBuilder
+reset(name, version, description)
+beginStage(spec)
+endStage()
+addTask(task)
+setAssignment(strategy)
+setRouting(source_id, strategy)
+setEscalationChain(head)
+addCancelSignal(name)
+getResult() : WorkflowDefinition
getResult()
<<Builder>>
StandardWorkflowBuilder
-root : unique_ptr<WorkItem>
-open_stages : vector<WorkItem*>
policies : DefinitionPolicies
WorkflowLibrary
-entries : map<string, Entry>
+add(pack_key, definition)
+get(name) : WorkflowDefinition
+packKeyOf(name)
+names()
<<Prototype>>
WorkflowDefinition
-name_ref : string
-description_ref : string
-version_ref : unsigned
-template_root : unique_ptr<WorkItem>
policies_ref : DefinitionPolicies
+validate()
+validationProblems()
+createInstance(id, data, services) : unique_ptr<WorkflowInstance>
closes the tree
<<Struct>>
DefinitionPolicies
+assignment : AssignmentStrategy
+routing : map<Id, RoutingStrategy>
+escalation : EscalationHandler
+cancel_signals : set<string>
_policies_ref
escalation
assignment
routing
engine/policies
<<Strategy>>
AssignmentStrategy
+chooseParticipant(item, role, directory, data) : Id
<<Strategy>>
RoutingStrategy
+decide(finished, outcome, data) : RoutingDecision
<<Struct>>
RoutingDecision
+activate : vector<Id>
+skip : vector<Id>
<<Strategy>>
RoleBasedAssignment
+chooseParticipant()
<<Strategy>>
DataDrivenAssignment
-data_key : string
-fallback : AssignmentStrategy
+chooseParticipant()
-successor
<<Strategy>>
DependencyRouting
+decide()
<<Strategy>>
OutcomeTableRouting
-rules : map<string, RoutingDecision>
-default_rule : RoutingDecision
+addRule(key, activate, skip)
+setDefault(activate, skip)
+decide()
<<Strategy>>
DataValueRouting
-key : string
-rules : map<string, RoutingDecision>
+addRule(value, activate, skip)
+setDefault(activate, skip)
+decide()
<<Chain of Responsibility>>
EscalationHandler
-successor : shared_ptr<EscalationHandler>
+handle(request) : EscalationResult
#canHandle(request) : bool
#resolve(request) : EscalationResult
<<Chain of Responsibility>>
RoleEscalationHandler
-approver_role : string
-level : unsigned
#canHandle(request)
#resolve(request)
<<Struct>>
EscalationRequest
+instance_id : Id
+item_id : Id
+reason : string
+level : unsigned
+data : WorkflowData*
<<Struct>>
EscalationResult
+resolved : bool
+approver_role : string
+handled_by : string
canHandle ? resolve : succeed
level matches
engine/runtime
<<Memento>>
WorkflowInstance
-id_ref : Id
-definition_name : string
-definition_version : unsigned
-status_ref : InstanceStatus
-data_ref : WorkflowData
-root_ref : unique_ptr<WorkItem>
-events_ref : EventPublisher
-audit_ref : AuditLog
-audit_observer
-context_ref : WorkContext
-flow : FlowController
-deadlines : DeadlineMonitor
-actions_ref : ActionHistory
+start()
+cancel(reason)
+findItem(id) : WorkItem*
+itemsFor(participant) : vector<WorkItem*>
+addObserver(observer)
+sweepDeadlines() : size_t
+captureSnapshot() : InstanceMemento
+restoreSnapshot(memento)
-finish(status, message)
<<Memento>>
InstanceMemento
-items : map<Id, WorkItemMemento>
-data : WorkflowData
-status : InstanceStatus
fixed FlowController
<<Struct>>
Services
+clock : Clock&
+directory : ParticipantDirectory&
+connectors : ConnectorRegistry&
<<Struct>>
AuditEntry
+when : TimePoint
+type : EventType
+item_id : Id
+actor_id : Id
+message : string
-flow
-deadlines
DeadlineMonitor
-instance : WorkflowInstance&
-clock : Clock&
-reported : set<Id>
+sweep() : size_t
publishes DeadlineMissed
<<Observer>>
<<Strategy>>
FlowController
-instance : WorkflowInstance&
-policies : DefinitionPolicies
-pending : deque<WorkflowEvent>
-draining : bool
+onEvent(event)
+releaseEligible()
-react(event)
-assignIfPossible(item)
-runAutomatic(item)
-applyDecision(decision)
-evaluateInstance()
queue, then drain
<<Observer>>
AuditObserver
-log : AuditLog&
+onEvent(event)
<<Observer>>
NotificationObserver
-connectors : ConnectorRegistry&
-connector_name : string
+onEvent(event)
AuditLog
-entries_ref : vector<AuditEntry>
+append(entry)
+entries()
+format(entry) : string
engine/events
<<Observer>>
EventObserver
+onEvent(event : WorkflowEvent)
<<Observer>>
EventSubject
+attach(observer)
+detach(observer)
+notify(event)
<<Observer>>
WorkflowEvent
-type_ref : EventType
-instance_id : Id
-item_id : Id
-actor_id : Id
-outcome_ref : Outcome
-message_ref : string
-count_ref : unsigned
-when_ref : TimePoint
+byActor(id)
+withOutcome(o)
+withMessage(m)
+withCount(n)
+type()
+itemId()
+outcome()
-observers
<<Observer>>
EventPublisher
-observers : vector<EventObserver*>
+attach(observer)
+detach(observer)
+notify(event)
+observerCount() : size_t
-events_ref
engine/commands
<<Command>>
ActionHistory
-instance : WorkflowInstance&
-executed : vector<unique_ptr<Command>>
+execute(command) : CommandResult
+undoLast() : CommandResult
+size()
+describeAll()
<<Command>>
Command
+execute() : CommandResult
+undo() : CommandResult
+isReversible() : bool
+describe() : string
executed
returns
<<Struct>>
CommandResult
+success : bool
+message : string
+value
+failureKind
<<Template Method>>
<<Memento>>
<<Command>>
InstanceCommand
+execute()
+undo()
#doExecute(item, actor)
#verb() : string
#detail() : string
<<Command>>
<<Composite>>
MacroCommand
-name : string
-children : vector<unique_ptr<Command>>
-executed_count : size_t
-executed : bool
+add(child)
+size()
+execute()
+undo()
children
inner_ref
<<Decorator>>
<<Command>>
CommandDecorator
#inner_ref : unique_ptr<Command>
+execute()
+undo()
+isReversible()
+describe()
forward to inner
<<Command>>
<<Decorator>>
RetryingCommand
-max_attempts : unsigned
+execute()
retry until success or max attempts
in order
in reverse order
<<Command>>
StartCommand
<<Command>>
SubmitCommand
<<Command>>
CompleteCommand
<<Command>>
ApproveCommand
<<Command>>
RejectCommand
<<Command>>
EscalateCommand
<<Command>>
CancelCommand
#doExecute(item, actor)
#verb()
#detail()
<<Command>>
CommandFactory
-creators : map<string, Creator>
+registerVerb(verb, creator)
+create(verb, request) : Command
+verbs() : vector<string>
<<Struct>>
CommandRequest
+instance : WorkflowInstance&
+item_id : Id
+actor : Participant&
+outcome : Outcome
+text : string
snapshot then doExecute
restores Snapshot
calls the matching WorkItem op
engine/items
<<Composite>>
<<Iterator>>
<<Prototype>>
<<Visitor>>
WorkItem
+spec() : WorkItemSpec&
+status() : ItemStatus
+assignee() : Id&
+children() : vector<WorkItem*>
+add(child) : WorkItem&
+find(id) : WorkItem*
+publish()
+assign(id)
+start(actor)
+submit(actor, outcome)
+complete(actor, outcome)
+approve(reviewer, outcome)
+reject(reviewer, reason)
+escalate(reason)
+cancel(reason)
+skip(reason)
+expire(outcome_code, reason)
+clone() : unique_ptr<WorkItem>
+accept(visitor)
+createIterator() : WorkItemIterator
+createMemento()
+restore()
+bind(context)
WorkItemSpec
-id_ref : string
-name_ref : string
-required_role : string
-reviewer_role : string
-dependencies_ref : vector<Id>
-review_required : bool
-automatic_ref : bool
-max_retries : unsigned
-due_after_seconds : long
-timeout_code : string
+requireRole()
+reviewedBy()
+dependsOn()
+reviewRequired()
+automatic()
+dueAfter()
+onTimeout()
+maxRetries()
createIterator()
accept(visitor)
delegate to state
-children_ref
-spec_ref
<<Iterator>>
WorkItemIterator
+first()
+next()
+isDone() : bool
+currentItem() : WorkItem&
<<Iterator>>
DepthFirstIterator
-order : vector<WorkItem*>
-position : size_t
+first()
+next()
+isDone()
+currentItem()
<<Visitor>>
WorkItemVisitor
+visitTask(task : TaskItem)
+visitStage(stage : StageItem)
<<Visitor>>
ProgressVisitor
-counts : map<ItemStatus, size_t>
-total_ref : size_t
+visitTask(task)
+visitStage(stage)
+total()
+count(status)
+percentSettled() : double
perform a site check, then sweep
<<Composite>>
StageItem
-spec_ref : WorkItemSpec
-children_ref : vector<unique_ptr<WorkItem>>
+status()
+add(child)
+children()
+clone()
+createIterator()
+bind(context)
derived from children
deep copy
bind all children
<<State>>
<<Template Method>>
<<Composite>>
TaskItem
-spec_ref : WorkItemSpec
-state : WorkItemState*
-context_ref : WorkContext*
-assignee_ref : Id
-assigned_at : TimePoint
-rejections_ref : unsigned
-failures_ref : unsigned
-escalations_ref : unsigned
-last_outcome : Outcome
-completed_by : Id
+doAll() : TimePoint
+restore(memento)
+createMemento()
-executeWork(actor, outcome) : bool
#performWork(actor, outcome)
#validateOutcome(outcome)
#onComplete(outcome)
#performIrreversibleWork() : bool
-component_ref
-spec_ref
-state
template
default throw InvalidStateError
<<Decorator>>
<<Composite>>
WorkItemDecorator
-component_ref : unique_ptr<WorkItem>
-context_ref : WorkContext*
#component()
#cloneComponent()
every WorkItem operation forwards to component_ref
<<Prototype>>
ClonableTask<Derived>
+clone() : unique_ptr<WorkItem>
run rules, then forward
<<Template Method>>
ManualTask
#performWork(actor, outcome)
<<Memento>>
WorkItemMemento
-status : ItemStatus
-assignee : Id
-assigned_at : TimePoint
-rejections : unsigned
-failures : unsigned
-escalations : unsigned
-last_outcome : Outcome
-completed_by : Id
<<Decorator>>
SecureWorkItem
+start()
+submit()
+complete()
+approve()
+reject()
<<Decorator>>
ValidateWorkItem
-rules : vector<ValidationRule>
+submit()
+complete()
-rules
<<Decorator>>
ValidationRule
+check(item, data, outcome) : string
<<Decorator>>
RequiredDataRule
-keys : vector<string>
+check(item, data, outcome)
<<State>>
WorkItemState
+status() : ItemStatus
+publish()
+assign()
+submit()
+complete()
+approve()
+reject()
+escalate()
+cancel()
+skip()
+expire()
-throwTo(item, next)
-item()
-state
ConcreteStates
CreatedState
AvailableState
AssignedState
InProgressState
UnderReviewState
EscalatedState
CompletedState
CancelledState
SkippedState
WorkContext
-instance_id : Id
-data_ref : WorkflowData&
-events_ref : EventPublisher&
-clock_ref : Clock&
-connectors_ref : ConnectorRegistry&
-directory_ref : ParticipantDirectory&
-irreversible : unsigned
+data()
+events()
+clock()
+connectors()
+directory()
+markIrreversible()
-context_ref
engine/core
<<enumeration>>
EventType
InstanceStarted
InstanceCompleted
InstanceFailed
InstanceCancelled
ItemAvailable
ItemAssigned
ItemStarted
ItemSubmitted
ItemCompleted
ItemRejected
ItemEscalated
ItemCancelled
ItemSkipped
ItemFailed
ItemReverted
DeadlineMissed
ExternalSignal
EngineNote
Clock
+now() : TimePoint
SystemClock
+now()
ManualClock
-current : TimePoint
+now()
+advance(seconds)
<<enumeration>>
OutcomeKind
Accepted
Rejected
Escalated
Failed
<<enumeration>>
InstanceStatus
Created
Running
Completed
Failed
Cancelled
Participant
-id_ref : Id
-display_name : string
-roles : set<string>
-attributes : map<string, string>
+hasRole(role) : bool
+setAttribute(key, value)
+attribute(key, fallback)
ParticipantDirectory
+add(participant)
+find(id)
+require(id)
+withRole(role) : vector<Participant>
WorkflowData
-values : map<string, string>
+has(key)
+get(key)
+getOr(key, fallback)
+set(key, value)
Outcome
+accepted()
+escalated()
+routingKey() : string
+rejected()
+failed()
InvalidStateError
PermissionDeniedError
EngineError
+retryable()
InvalidDefinitionError
UnsupportedOperationError
ExternalFailure
NotFoundError
ValidationError
extends std::runtime_error
<<enumeration>>
ItemStatus
Created
Available
Assigned
InProgress
UnderReview
Escalated
Completed
Cancelled
Skipped
engine/connectors
<<Adapter>>
ExternalConnector
+name() : string
+invoke(request) : ConnectorResult
-connectors
ConnectorRegistry
-connectors : map<string, ExternalConnector>
+register(connector)
+has(name) : bool
+require(name) : ExternalConnector&
<<Struct>>
ConnectorResult
+success : bool
+retryable : bool
+message : string
+payload : map<string, string>
+ok(msg, payload)
+failure(msg, retryable)
<<Adapter>>
LoggingNotifier
-sent_ref : vector<string>
+invoke(request)
+sent() : vector<string>
<<Struct>>
CommandRequest
+operation : string
+params : map<string, string>
Legend
fill = main pattern · hexagon dots = every collaboration the class is in
Abstract Factory
Builder
Prototype
Composite
Decorator
Adapter
Facade
State
Template Method
Strategy
Chain of Responsibility
Observer
Command
Memento
Iterator
Visitor
RELATIONSHIPS
Inheritance (is-a)
Composition (owns, lifetime bound)
Aggregation (has, non-owning)
Association (knows)
Dependency (uses / creates)
OTHER SHAPES
Plain class / struct (no pattern)
Enumeration
Note
Package (one per folder)
PACKAGE ARROWS
Coloured dashed arrow between packages: the colour is the package being used.
engine/core
engine/commands
engine/connectors
engine/runtime
+ public # protected - private
italic = abstract
underline = static
Packages correspond to the folders in the system. The class diagram above outlines the significant classes, operations, attributes and interactions of the industry-agnostic TIVIDY engine.

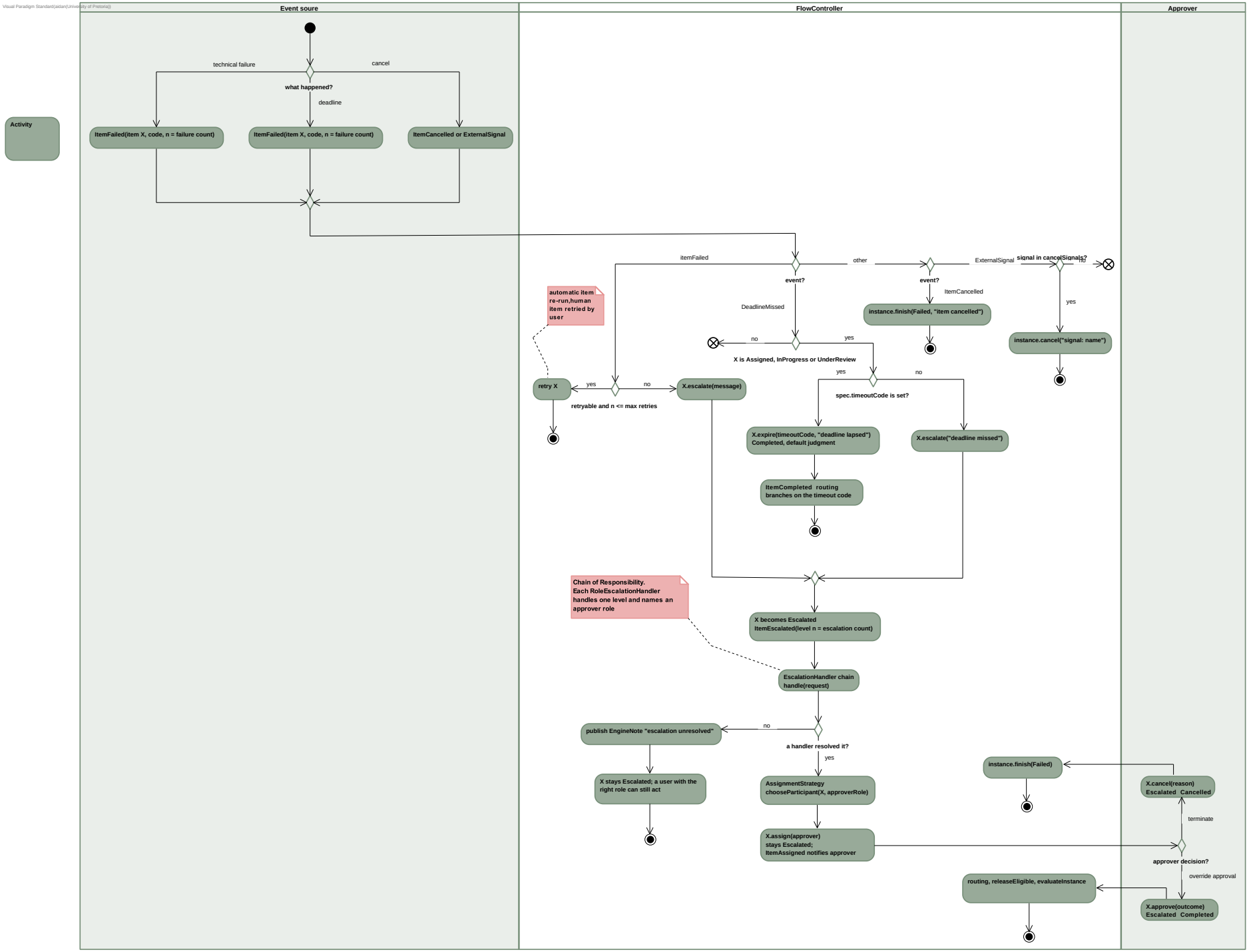

Visual Paradigm Standard(adan@University of Pretoria))
<<Abstract Factory>>
IndustryPack
+key()
+displayName() : string
+createProfile() : ClientProfile
+createParticipants() : vector<Participant>
+createEscalationChain() : shared_ptr<const EscalationHandler>
+createDirector() : WorkflowDirector
+createConnectors() : vector<ExternalConnector>
+createAssignmentStrategy() : shared_ptr<const AssignmentStrategy>
<<Builder>>
WorkflowDirector
+build(builder) : WorkflowDefinition
+workflowName()
+version()
+description() : string
+construct(builder)
construct(builder)
<<Builder>>
WorkflowBuilder
+reset(name, version, description)
+beginStage(spec)
+endStage()
+addTask(task)
+setAssignment(strategy)
+setRouting(source_id, strategy)
+setEscalationChain(head)
+addCancelSignal(name)
+getResult() : WorkflowDefinition
<<Strategy>>
OutcomeTableRouting
-rules : map<string, RoutingDecision>
-default_rule : RoutingDecision
+addRule(key, activate, skip)
+setDefault(activate, skip)
+decide()
<<Strategy>>
DataValueRouting
-key : string
-rules : map<string, RoutingDecision>
+addRule(value, activate, skip)
+setDefault(activate, skip)
+decide()
<<Decorator>>
SecureWorkItem
+start()
+submit()
+complete()
+approve()
+reject()
<<Decorator>>
ValidatedWorkItem
-rules : vector<ValidationRule>
+submit()
+complete()
ConcreteFactory
Director
<<Builder>>
ServiceActivationDirector
-pack : IndustryPack&
+workflowName() : "service-activation"
+version() : 1
+construct(builder)
+description() : string
createDirector()
<<Decorator>>
RequiredDataRule
-keys : vector<string>
+check(item, data, outcome)
<<Abstract Factory>>
TelecomPack
+key() : "telecom"
+createDirector() : ServiceActivationDirector
+createParticipants() : vector<Participant>
+createConnectors() : vector<unique_ptr<ExternalConnector>>
+createAssignmentStrategy() : shared_ptr<const AssignmentStrategy>
+createEscalationChain() : shared_ptr<const EscalationHandler>
+createProfile() : ClientProfile
+displayName() : string
CreditBureauAdapter, NetworkProvisioningAdapter default LoggingNotifier
RegionalTechnicianAssignment
network-engineer (1) → noc-manager (2)
<<Struct>>
ClientProfile
+product_name : string
+tagline : string
+banner : string
+accent : string
+vocabulary : map<string, string>
+term(engine_term) : string
<<Chain of Responsibility>>
RoleEscalationHandler
-approver_role : string
-level : unsigned
+scanHandle(request)
#resolve(request)
<<Strategy>>
RegionalTechnicalAssignment
+chooseParticipant(item, role, directory, data) : Id
ConcreteStrategy
<<Strategy>>
AssignmentStrategy
+chooseParticipant(item, role, directory, data) : Id
<<Strategy>>
RoleBasedAssignment
+chooseParticipant()
field technician is picked by region, other roles are picked with RoleBasedAssignment
<<Adapter>>
CreditBureauAdapter
-vendor : unique_ptr<CreditBureauVendor>
+name() : string
+invoke(request) : ConnectorResult
<<Adapter>>
CreditBureauVendor
-failures_left : unsigned
+failNextCalls(n)
+score(id_number) : int
1
-vendor
operation is score(id_number)
<<Adapter>>
ExternalConnector
+name() : string
+invoke(request) : ConnectorResult
<<Adapter>>
NetworkProvisioningAdapter
-vendor : unique_ptr<NetworkVendor>
+name() : string
+invoke(request) : ConnectorResult
<<Adapter>>
NetworkVendor
-failures_left : unsigned
-provisioned : string
-active : string
+failNextCalls(n)
+provision(service_ref, product)
+activate(service_ref)
+isActive(service_ref) : bool
operations are provision · activate
Adaptes
calls connector "network-provisioning"
<<Template Method>>
ManualTask
#performWork(actor, outcome)
telecom_task_classes
<<Template Method>>
<<Prototype>>
CreditCheckTask
#performWork(actor, outcome)
<<Template Method>>
<<Prototype>>
ProvisionServiceTask
#performWork(actor, outcome)
#performsIrreversibleWork() : true
calls connector "credit-bureau"
<<Prototype>>
CloneableTask<Derived>
+clone() : unique_ptr<WorkItem>
<<State>>
<<Template Method>>
<<Composite>>
TaskItem
-spec_ref : WorkItemSpec
-state : WorkItemState*
-context_ref : WorkContext*
-assignee_ref : Id
-assigned_at : TimePoint
-rejections_ref : unsigned
-failures_ref : unsigned
-escalations_ref : unsigned
-last_outcome : Outcome
-completed_by : Id
+dueAt() : TimePoint
+restore(memento)
+createMemento()
+executeWork(actor, outcome) : bool
#performWork(actor, outcome)
#validateOutcome(outcome)
#onCompleted(outcome)
#performsIrreversibleWork() : bool
Legend
fill = main pattern · hexagon dots = every collaboration the class is in
PATTERNS IN THIS DIAGRAM
Abstract Factory
Adapter
Template Method
Composite
Decorator
Builder
Prototype
State
Strategy
Chain of Responsibility
RELATIONSHIPS
Inheritance (is-a)
Composition (owns)
Dependency (uses / creates)
SHAPES
Solid border: this industry's own class
Dashed border: engine class (context)
White: plain class / struct
Note
Package
+ public # protected - private
italic = abstract
underline = static
The class diagram above outlines the significant classes, operations, attributes and interactions of the TIVIDY telecom engine.

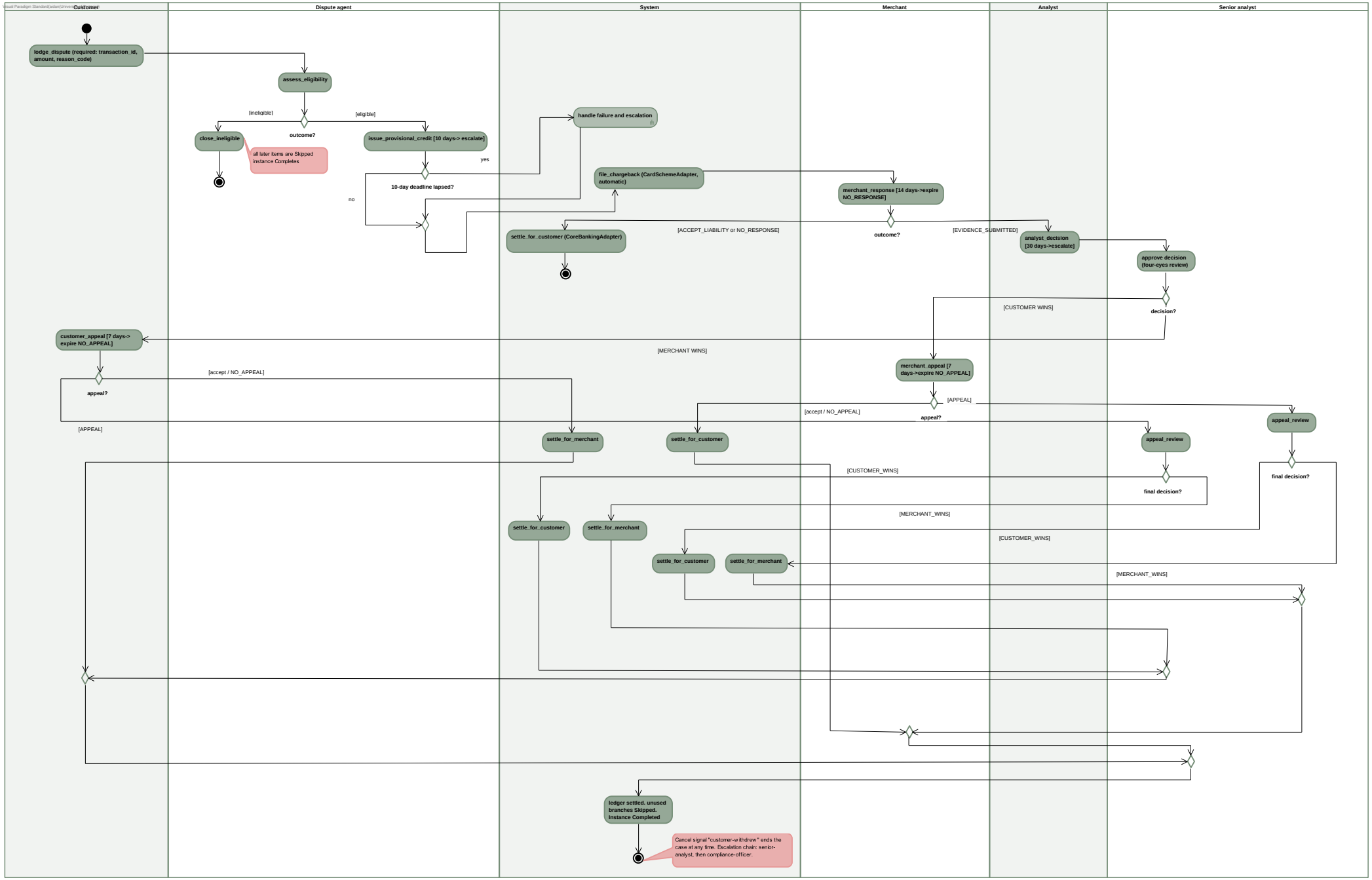
Visual Paradigm Standard(aidan(University of Pretoria))

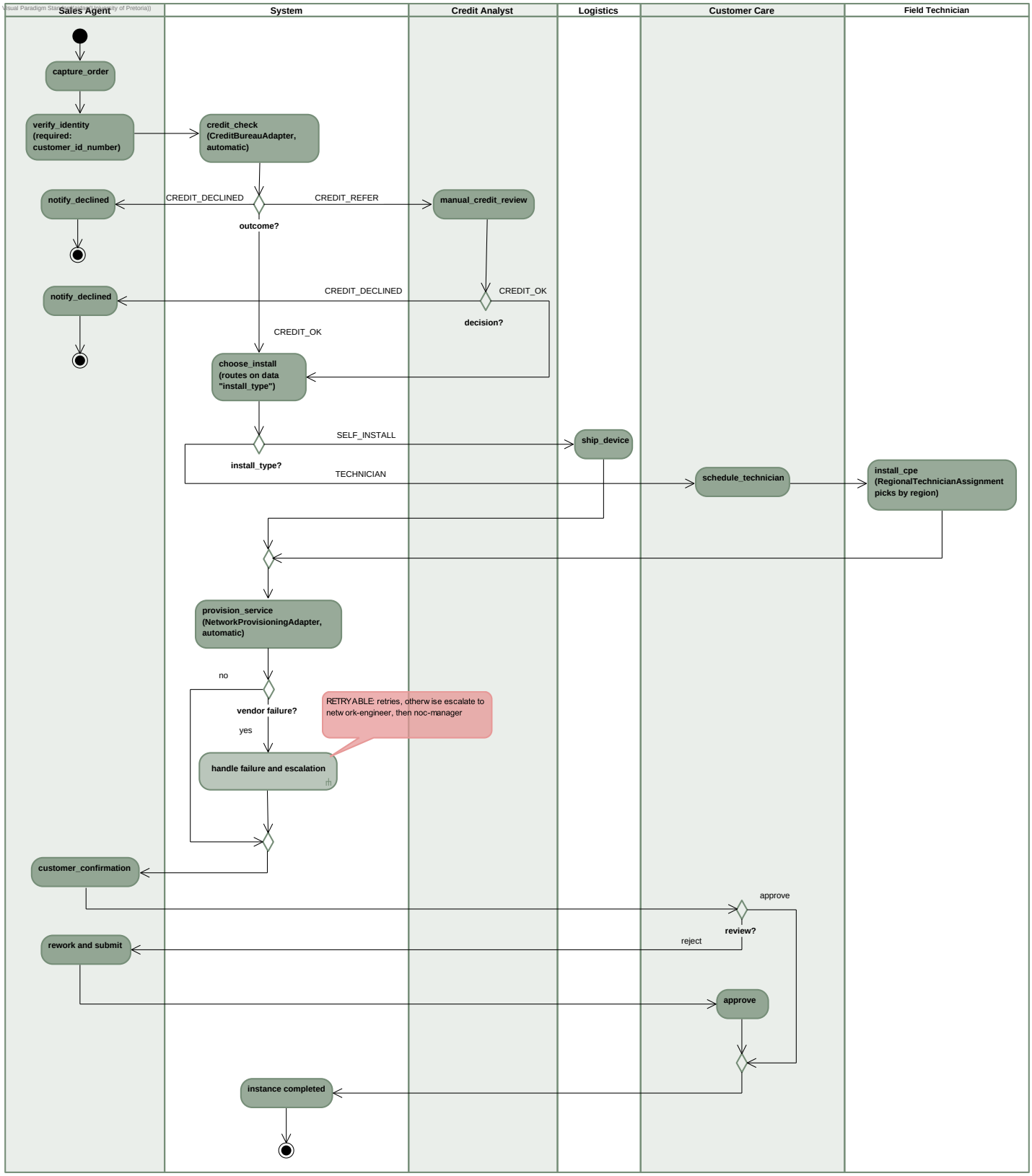



Visual Paradigm Standard(jaidan(University of Pretoria))
Tividy (Facade)
FlowController
Participant
Reviewer
clone the template tree (Prototype)
startWorkflow(name, data)
definition_createInstance()
Created -> Running publish InstanceStarted
instance_start()
releaseEligible()
every Created item whose dependencies (item and ancestor stages) are settled -> publish
any other item whose dependencies are settled assigned independently
fork: items that share a satisfied dependency become available together and are worked in parallel
ItemAvailable(item X)
ItemAvailable(item Y)
assignIfPossible()
AssignmentStrategy.chooseParticipant(role)
join: a stage only settles once all of its children have settled
ItemAssigned
Assigned -> InProgress
yes
no
item is automatic?
start(actor)
runAutomatic()
do the work
start(system)
may call an ExternalConnector (Adapter)
no
requires review?
yes
performWork()
InProgress -> Completed
complete(outcome)
submit(outcome)
InProgress -> UnderReview
reject
approve
reviewer decision?
rejections < maxRetries
yes
no
ItemCancelled event
approve(outcome)
reject(reason)
item cancelled
back to InProgress
instance_finish(Failed)
rework and submit
approve(outcome)
itemCompleted
routing.decision(item, outcome, data)
activate chosen branches, skip the rest
applyDecision()
releaseEligible()
evaluateInstance()
yes
root stage not settled?
no
Completed
Cancelled
root status?
instance.finish(Completed)
instance.finish(Failed)

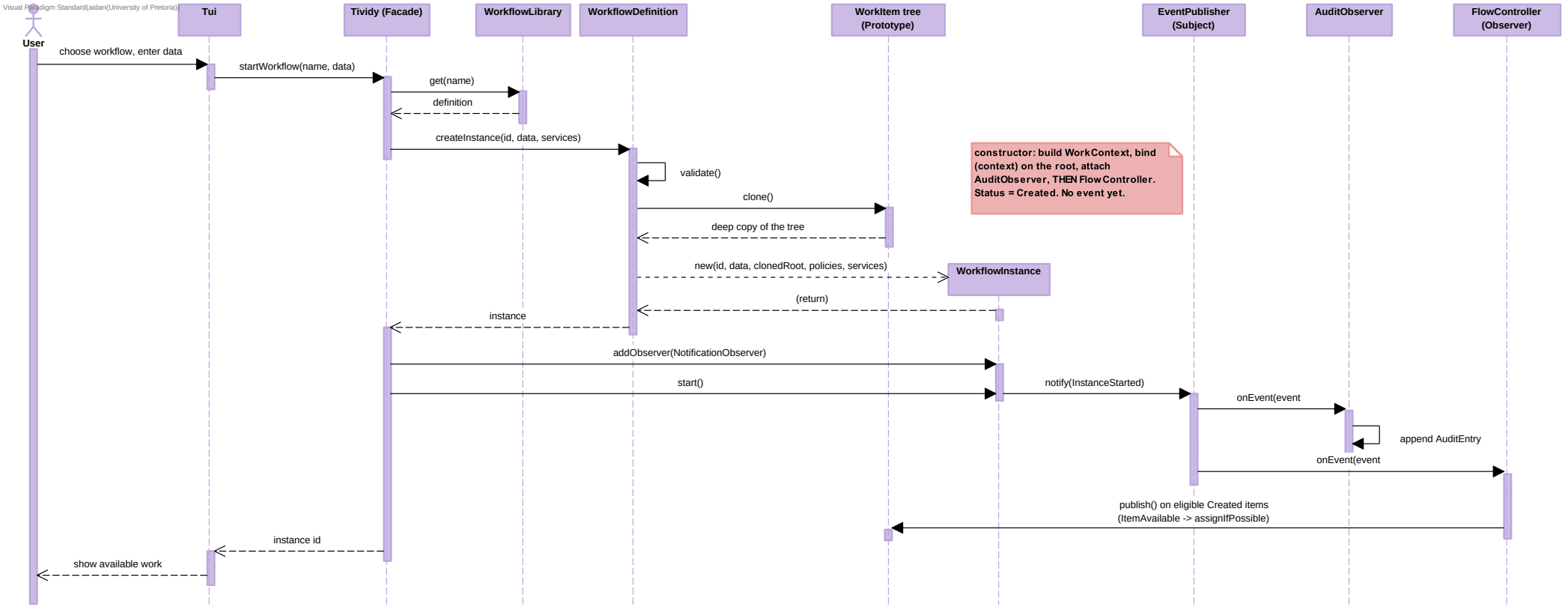
Visual Paradigm Standard(aidan(University of Pretoria))

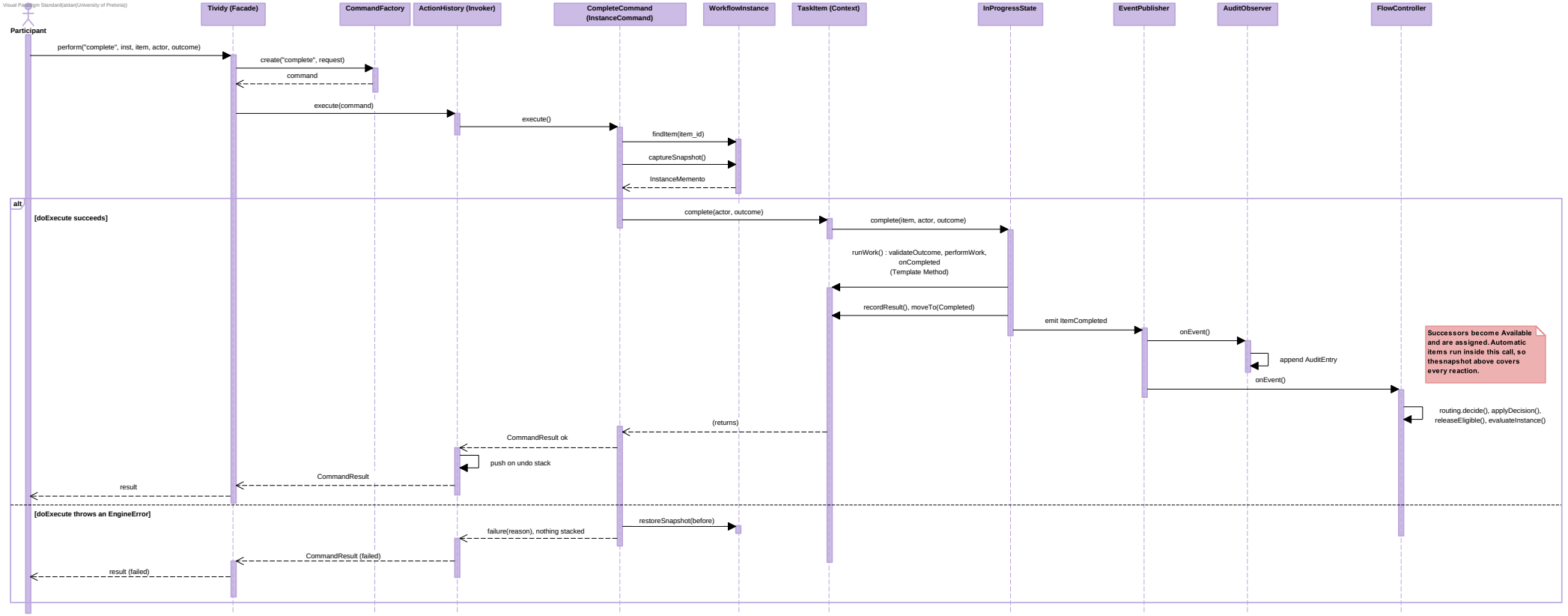


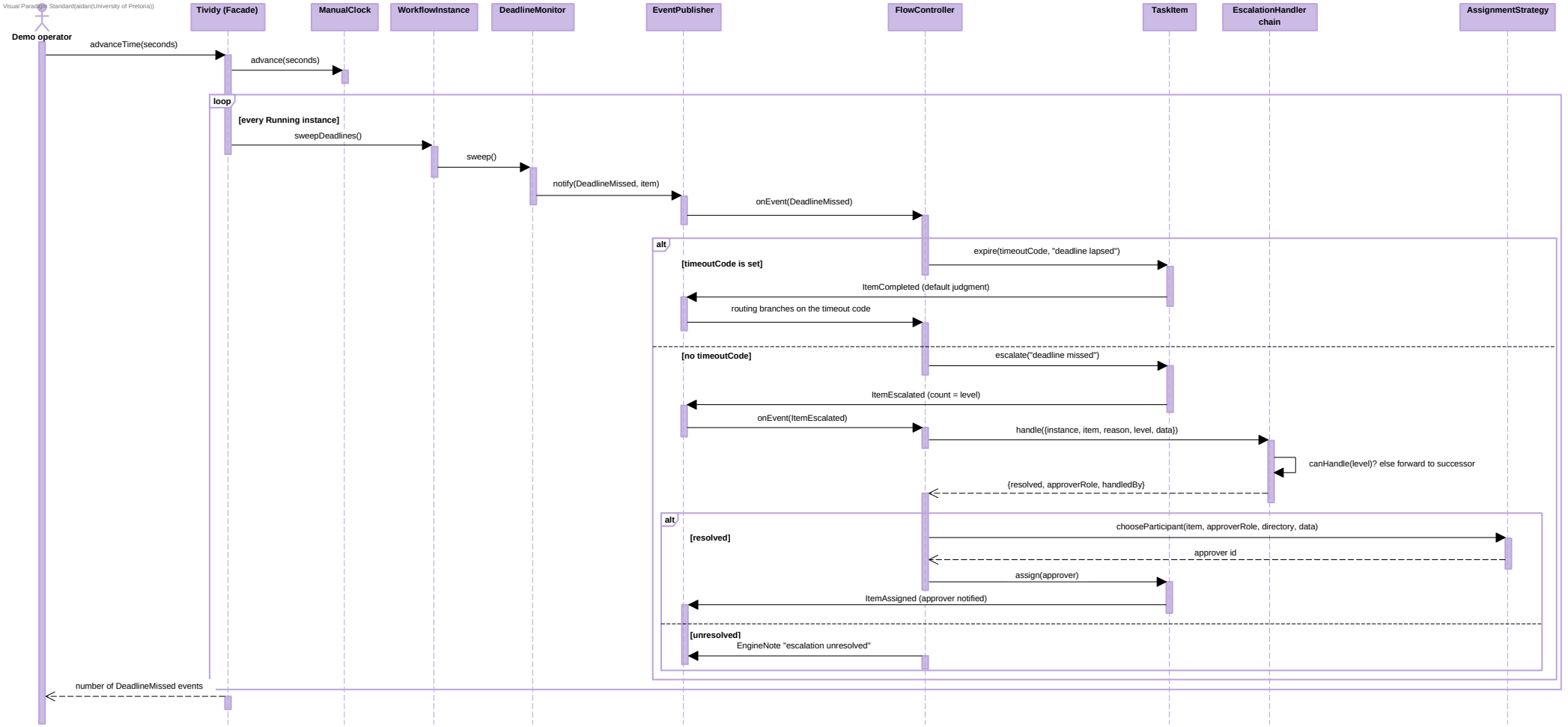

Customer
Dispute agent
System
Merchant
Analyst
Senior analyst
lodge_dispute (required: transaction_id, amount, reason_code)
assess_eligibility
[ineligible]
[eligible]
outcome?
close_ineligible
all later items are Skipped instance Completes
issue_provisional_credit [10 days-> escalate]
yes
10-day deadline lapsed?
no
handle failure and escalation
file_chargeback (CardSchemeAdapter, automatic)
merchant_response [14 days->expire NO_RESPONSE]
outcome?
[ACCEPT_LIABILITY or NO_RESPONSE]
[EVIDENCE_SUBMITTED]
settle_for_customer (CoreBankingAdapter)
analyst_decision [30 days->escalate]
approve decision (four-eyes review)
decision?
[CUSTOMER WINS]
[MERCHANT WINS]
customer_appeal [7 days-> expire NO_APPEAL]
merchant_appeal [7 days->expire NO_APPEAL]
[APPEAL]
appeal?
[accept / NO_APPEAL]
appeal?
[APPEAL]
[accept / NO_APPEAL]
appeal_review
appeal_review
final decision?
final decision?
settle_for_merchant
settle_for_customer
[CUSTOMER_WINS]
[MERCHANT_WINS]
settle_for_customer
settle_for_merchant
settle_for_customer
settle_for_merchant
[CUSTOMER_WINS]
[MERCHANT_WINS]
ledger settled. unused branches Skipped. Instance Completed
Cancel signal "customer-withdrew" ends the case at any time. Escalation chain: senior-analyst, then compliance-officer.


Sales Agent
System
Credit Analyst
Logistics
Customer Care
Field Technician
capture_order
verify_identity (required: customer_id_number)
credit_check (CreditBureauAdapter, automatic)
notify_declined
CREDIT_DECLINED
CREDIT_REFER
outcome?
manual_credit_review
notify_declined
CREDIT_DECLINED
CREDIT_OK
decision?
CREDIT_OK
choose_install (routes on data "install_type")
SELF_INSTALL
install_type?
ship_device
TECHNICIAN
schedule_technician
install_cpe (RegionalTechnicianAssignment picks by region)
provision_service (NetworkProvisioningAdapter, automatic)
no
vendor failure?
yes
RETRYABLE: retries, otherwise escalate to network-engineer, then noc-manager
handle failure and escalation
customer_confirmation
approve
review?
reject
rework and submit
approve
instance completed

Visual Paradigm Standard(aidan(University of Pretoria))




Visual Paradigm Standard(aidan(University of Pretoria))
Participant
Tividy (Facade)
CommandFactory
ActionHistory (Invoker)
CompleteCommand (InstanceCommand)
WorkflowInstance
TaskItem (Context)
InProgressState
EventPublisher
AuditObserver
FlowController
perform("complete", inst, item, actor, outcome)
create("complete", request)
command
execute(command)
execute()
findItem(item_id)
captureSnapshot()
InstanceMemento
alt
[doExecute succeeds]
complete(actor, outcome)
complete(item, actor, outcome)
runWork() : validateOutcome, performWork, onCompleted
(Template Method)
recordResult(), moveTo(Completed)
emit ItemCompleted
onEvent()
append AuditEntry
onEvent()
Successors become Available and are assigned. Automatic items run inside this call, so thesnapshot above covers every reaction.
routing.decide(), applyDecision(), releaseEligible(), evaluateInstance()
(returns)
CommandResult ok
push on undo stack
CommandResult
result
[doExecute throws an EngineError]
restoreSnapshot(before)
failure(reason), nothing stacked
CommandResult (failed)
result (failed)


Visual Paradigm Standard(aidan(University of Pretoria))
Demo operator
Tividy (Facade)
ManualClock
WorkflowInstance
DeadlineMonitor
EventPublisher
FlowController
TaskItem
EscalationHandler chain
AssignmentStrategy
advanceTime(seconds)
advance(seconds)
loop
[every Running instance]
sweepDeadlines()
sweep()
notify(DeadlineMissed, item)
onEvent(DeadlineMissed)
alt
[timeoutCode is set]
expire(timeoutCode, "deadline lapsed")
ItemCompleted (default judgment)
routing branches on the timeout code
[no timeoutCode]
escalate("deadline missed")
ItemEscalated (count = level)
onEvent(ItemEscalated)
handle({instance, item, reason, level, data})
canHandle(level)? else forward to successor
{resolved, approverRole, handledBy}
alt
[resolved]
chooseParticipant(item, approverRole, directory, data)
approver id
assign(approver)
ItemAssigned (approver notified)
[unresolved]
EngineNote "escalation unresolved"
number of DeadlineMissed events